

SPRING BOOT LEARNING GUIDE

Weather App Codebase Explained

A practical mental model of the existing Spring Boot project: REST controllers, services, DTO mapping, external Weather API calls, Redis caching, configuration, and request flow.

Table of Contents

- | | |
|---|---------------------------------|
| 1. High-level overview | 7. External Weather API mapping |
| 2. Project and package structure | 8. Redis caching |
| 3. Important classes | 9. Configuration |
| 4. Class dependencies | 10. Gradle dependencies |
| 5. Spring Boot architecture | 11. Patterns used |
| 6. Request trace: GET /api/weather/Alexandria | 12. Architecture diagram |
| | 13. 10 concepts to learn |

1. High-Level Overview

This project is a Spring Boot REST API that returns weather forecast data for a city. A client calls `GET /api/weather/{city}`, the app requests forecast data from the Visual Crossing Weather API, maps that response into simpler DTOs, returns JSON, and caches the result in Redis for 10 minutes.

Controller

Receives HTTP requests and delegates work.

Service

Calls the external API and maps weather data.

DTOs

Shape incoming and outgoing JSON data.

Redis

Stores repeated weather results temporarily.

2. Project and Package Structure

```
weather-app
|-- build.gradle
|-- settings.gradle
|-- src
|   |-- main
|   |   |-- java/com/example/weather/app
|   |   |   |-- WeatherAppApplication.java
|   |   |   |-- config
|   |   |   |   |-- RedisConfig.java
|   |   |   |-- controller
|   |   |   |   |-- weatherController.java
|   |   |   |   |-- CacheController.java
|   |   |   |-- dto
|   |   |   |   |-- WeatherApiResponse.java
|   |   |   |   |-- WeatherResponseDTO.java
|   |   |   |   |-- HourlyForecastDTO.java
|   |   |   |   |-- DailyForecastDTO.java
|   |   |   |   |-- LocationDTO.java
|   |   |   |-- model
|   |   |   |-- repository
|   |   |   |-- service
|   |   |   |   |-- WeatherService.java
|   |   |   |   |-- RedisCacheService.java
|   |   |-- resources
|   |   |   |-- application.properties
|   |   |   |-- static
|   |   |-- templates
|   |-- test/java/com/example/weather/app
|       |-- WeatherAppApplicationTests.java
```

Package / Folder	Responsibility
<code>com.example.weather.app</code>	Root package. Spring scans this package and its subpackages.
<code>config</code>	Infrastructure configuration, currently Redis cache setup.
<code>controller</code>	REST API layer. Receives HTTP requests and returns responses.
<code>service</code>	Business logic layer. Calls external APIs, maps data, and provides Redis helper logic.
<code>dto</code>	Data Transfer Objects used for external API input shape and internal API response shape.
<code>model</code>	Empty right now. Usually used for domain/database entities.
<code>repository</code>	Empty right now. No database repository layer is currently implemented.
<code>resources</code>	Application properties and optional static/template resources.

3. Important Classes

WeatherAppApplication

What: The application entry point. **Why:** Spring Boot needs a bootstrap class. **How:** `SpringApplication.run(...)` starts the Spring container and embedded server.

`@SpringBootApplication` enables auto-configuration and component scanning. `@EnableCaching` enables annotation-based caching, which makes `@Cacheable` work.

weatherController

What: REST controller for weather endpoints. **Why:** It exposes weather functionality through HTTP. **How:** The base path is `/api/weather`. The method `getWeather(@PathVariable String city)` handles `GET /api/weather/{city}`.

WeatherService

What: Main business service. **Why:** It keeps external API calls, mapping, temperature conversion, and caching away from the controller. **How:** It reads configuration with `@Value`, calls Visual Crossing with `RestTemplate`, maps the result into `WeatherResponseDTO`, and returns it.

RedisConfig

What: Redis cache configuration. **Why:** The app needs to define how cached values are stored and how long they live. **How:** It creates a `RedisCacheManager` with a 10-minute TTL and JSON value serialization.

CacheController and RedisCacheService

What: A small debugging path for viewing Redis keys. **Why:** It helps you see what cache entries exist. **How:** `GET /cache/keys` calls `RedisCacheService.getAllKeys()`, which uses `RedisTemplate.keys("*")`.

DTO Classes

DTO	Purpose
<code>WeatherApiResponse</code>	Represents the JSON shape returned by Visual Crossing.
<code>WeatherResponseDTO</code>	Represents the clean JSON response returned by your API.
<code>HourlyForecastDTO</code>	Represents one hourly forecast item.
<code>DailyForecastDTO</code>	Represents one daily forecast item.
<code>LocationDTO</code>	Currently unused.

4. Class Dependencies

```
weatherController
  -> WeatherService

WeatherService
  -> WeatherApiResponse
  -> WeatherResponseDTO
  -> HourlyForecastDTO
  -> DailyForecastDTO
  -> RestTemplate
  -> application.properties values

CacheController
  -> RedisCacheService

RedisCacheService
  -> RedisTemplate<String, Object>

RedisConfig
  -> RedisConnectionFactory
  -> creates RedisCacheManager
  -> creates RedisTemplate
```

The important idea: most objects are not manually created by your own code. Spring creates them, manages them as beans, and injects them where needed.

5. Spring Boot Architecture

Controllers

What: Classes that handle HTTP requests. **Why:** They form the public API boundary. **How here:** `weatherController` handles weather routes and `CacheController` handles cache key inspection.

Services

What: Classes that contain business logic. **Why:** Controllers should stay small and delegate real work. **How here:** `WeatherService` owns weather fetching and mapping. `RedisCacheService` owns direct Redis access for key lookup.

DTOs

What: Simple objects used to transfer data. **Why:** They keep external API shape separate from your own API response shape. **How here:** `WeatherApiResponse` is the external shape; `WeatherResponseDTO` is your API response shape.

Dependency Injection and Beans

What: Spring provides objects to classes automatically. **Why:** It reduces manual setup and lets Spring manage lifecycles, proxies, and configuration. **How here:** `@Service`, `@RestController`, `@Configuration`, and `@Bean` all create Spring-managed objects.

Repositories

The `repository` package exists but is empty. There is no database access layer in the current application.

6. Request Trace: GET /api/weather/Alexandria

1. The HTTP request enters Spring Boot's embedded web server.
2. Spring MVC finds `weatherController` because it has `@RequestMapping("/api/weather")`.
3. Spring matches `@GetMapping("/{city}")`.
4. Spring extracts `Alexandria` from the URL and passes it as `city`.
5. The controller calls `weatherService.getWeather(city)`.
6. Because `getWeather` has `@Cacheable`, Spring checks Redis first.
7. The cache key is calculated as `city.toLowerCase()`, so `Alexandria` becomes `alexandria`.
8. If Redis has the value, Spring returns it without running the method body.
9. If Redis does not have the value, `WeatherService.getWeather("Alexandria")` runs.
10. The service builds the Visual Crossing URL and calls it using `RestTemplate`.
11. Jackson maps the external JSON into `WeatherApiResponse`.
12. The service maps that object into `WeatherResponseDTO`.
13. The response is cached in Redis for 10 minutes.
14. Spring MVC serializes the DTO to JSON and returns it to the client.

7. External Weather API Mapping

The external API call is built from values in `application.properties`:

```
String url = baseUrl + "/" + city + "?key=" + apiKey;
```

Then the service calls:

```
WeatherApiResponse weatherApiResponse =
    restTemplate.getForObject(url, WeatherApiResponse.class);
```

Spring uses Jackson to convert JSON fields into Java fields with matching names. The service then performs manual mapping:

External Field	Internal DTO Field
<code>resolvedAddress</code>	<code>WeatherResponseDTO.location</code>
<code>days[0].hours[].datetime</code>	<code>HourlyForecastDTO.time</code>
<code>days[0].hours[].temp</code>	<code>HourlyForecastDTO.temp</code> , converted to Celsius
<code>days[0].hours[].conditions</code>	<code>HourlyForecastDTO.condition</code>
<code>days[].datetime</code>	<code>DailyForecastDTO.date</code>
<code>days[].tempmin/tempmax/temp</code>	Daily temperature fields, converted to Celsius
<code>days[].conditions</code>	<code>DailyForecastDTO.condition</code>

8. Redis Caching

Where Redis is configured

Redis host and port are in `application.properties`. Cache behavior is in `RedisConfig`.

```
spring.data.redis.host=localhost  
spring.data.redis.port=6379  
spring.cache.type=redis
```

How Spring connects to Redis

Because the project includes `spring-boot-starter-data-redis`, Spring Boot auto-creates a `RedisConnectionFactory` from the Redis properties. `RedisConfig` uses that connection factory to create the cache manager and Redis template.

What `@Cacheable` does

`@Cacheable` wraps the method with caching behavior. Before the method body runs, Spring checks the cache. If a value exists, it returns the cached value. If not, the method runs and the result is stored.

```
@Cacheable(value = "weather", key = "#city.toLowerCase()")
```

Cache key, hit, and miss

- **Cache name:** `weather`
- **Method key:** lowercase city name
- **For Alexandria:** `alexandria`
- **Typical Redis key:** `weather::alexandria`
- **Cache hit:** Redis returns the cached DTO and the external API is not called.
- **Cache miss:** The service calls Visual Crossing, maps the data, stores the result, and returns it.

9. Configuration

Property	Purpose	Used By
<code>spring.application.name</code>	Names the application.	Spring Boot metadata/logging.
<code>weather.api.base-url</code>	Base URL for Visual Crossing.	<code>WeatherService.baseUrl</code> using <code>@Value</code> .
<code>weather.api.key</code>	API key for Visual Crossing.	<code>WeatherService.apiKEY</code> using <code>@Value</code> .
<code>spring.data.redis.host</code>	Redis hostname.	Spring Boot Redis auto-configuration.
<code>spring.data.redis.port</code>	Redis port.	Spring Boot Redis auto-configuration.
<code>spring.cache.type</code>	Selects Redis as cache provider.	Spring Cache auto-configuration.
<code>logging.level.org.springframework.cache</code>	Enables detailed cache logs.	Spring logging system.

In production, API keys are usually not stored directly in committed property files. They are usually supplied through environment variables, secret managers, or deployment configuration.

10. Gradle Dependencies

Dependency	Why it exists
<code>spring-boot-starter-web</code>	Provides Spring MVC, embedded server, REST controller support, JSON serialization, and HTTP client basics.
<code>spring-boot-starter-data-redis</code>	Provides Redis integration, <code>RedisTemplate</code> , Redis connection factory, and cache support.
<code>lombok</code>	Generates getters and setters for DTOs at compile time.
<code>spring-boot-starter-test</code>	Provides Spring Boot testing tools and JUnit support.
<code>junit-platform-launcher</code>	Runtime support for launching JUnit tests.

11. Design and Architectural Patterns

- **Layered architecture:** Controller -> Service -> External API/Redis.
- **DTO pattern:** Separate objects for external API input shape and your API output shape.
- **Dependency Injection:** Spring provides services and infrastructure objects.
- **Configuration pattern:** Properties and configuration classes separate settings from logic.
- **Proxy-based caching:** Spring wraps the service method and adds cache behavior around it.

- **Adapter / mapper style:** The service adapts Visual Crossing's response into your own response model.

12. Architecture Diagram

```

Client
|
| GET /api/weather/Alexandria
v
Spring MVC Dispatcher
|
v
weatherController
|
| calls getWeather(city)
v
Spring Cache Proxy
|
| checks key weather::alexandria
v
Redis
|
| cache miss
v
WeatherService
|
| RestTemplate HTTP GET
v
Visual Crossing Weather API
|
| JSON response
v
WeatherApiResponse
|
| manual mapping
v
WeatherResponseDTO
|
v
JSON response to Client

```

13. 10 Concepts to Learn, Beginner to Advanced

1. Java package organization and project structure.
2. Spring Boot startup with `@SpringBootApplication`.
3. REST endpoints with `@RestController`, `@RequestMapping`, `@GetMapping`, and `@PathVariable`.
4. Service classes with `@Service`.
5. Dependency Injection and the difference between field injection and constructor injection.
6. DTOs and why APIs use separate request/response objects.
7. Calling external APIs with `RestTemplate`.
8. Mapping external JSON into Java objects and then into your own DTOs.
9. Application configuration with `application.properties` and `@Value`.

10. Redis caching with `@EnableCaching`, `@Cacheable`, cache keys, TTL, and `RedisCacheManager`.

Generated for the existing codebase in `D:\Projects\springboot\project1\weather-app-spring\weather-app`. This document explains the current project as inspected; it does not change application behavior.